# Seed Lab Report: RSA Public-Key Encryption and Signature Lab

**Date:** 11th November, 2026

**Environment:** SEED Ubuntu VM

## 1. Overview

RSA (Rivest-Shamor-Adleman) The RSA cryptosystem is a public-key algorithm based on modular exponentiation over large prime factors.

RSA uses a key pair: a public key for encryption/verification and a private key for decryption/signing.

Because RSA key sizes range from 512 to 4096+ bits, operations exceed 64-bit primitive types and require arbitrary-precision Big Number libraries like OpenSSL `BIGNUM`.

## 2. Environment

GCC, OpenSSL (BIGNUM Library), Python 3

## 3. Task Implementation and Observations

This section contains secrenshots and a detailed explanation of the actions taken.

### Task 1: Deriving the Private Key

Given the prime numbers p, q and public exponent e that is relatively prime to $\phi(n)$ (euler's totient of n)

```
p = F7E75FDC469067FFDC4E847C51F452DF
q = E85CED54AF57E53E092113E62F436F4F
n = p x q
e = 0188C3
```

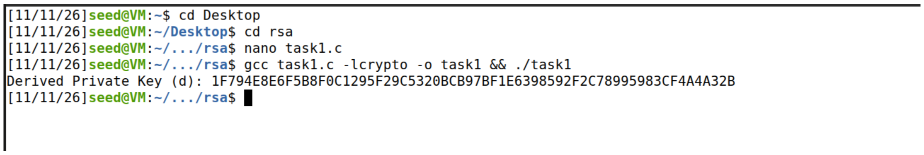
d, the private key is the modular inverse of e modulo $\phi(n)$.

It can also be represented as

```
e * d   1 (mod  (n))
```

The `BIGNUM` representation of this is

```
BN_mod_inverse(d, e, phi, ctx);
```



```
[11/11/26]seed@VM:~$ cd Desktop
[11/11/26]seed@VM:~/Desktop$ cd rsa
[11/11/26]seed@VM:~/.../rsa$ nano task1.c
[11/11/26]seed@VM:~/.../rsa$ gcc task1.c -lcrypto -o task1 && ./task1
Derived Private Key (d): 1F794E8E6F5B8F0C1295F29C5320BCB97BF1E6398592F2C78995983CF4A4A32B
[11/11/26]seed@VM:~/.../rsa$ █
```

Private key d that has been generated

### Task 2: Encrypting a Message

Let (e, n) be the public key. The message to be encrypted is `A top secret!`. The ASCII string is first converted to Hex using python.

```
$ python3-c 'print("A top secret!".encode("utf-8").hex())'
4120746f702073656372657421
```

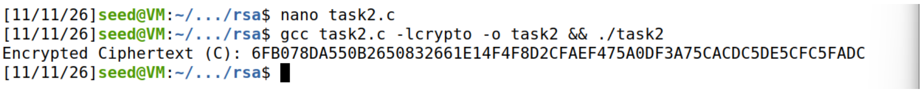
message converted to Hex

```
n = DCBFFE3E51F62E09CE7032E2677A78946A849DC4CDDE3A4D0CB81629242FB1A5
e = 010001 (this hex value equals to decimal 65537)
M = A top secret!
M(Hex) = 4120746f702073656372657421
d = 74D806F9F3A62BAE331FFE3F0A68AFE35B3D2E4794148AACBC26AA381CD7D30D
```

The public keys together with d for verification

```
C = M^e mod n
```

Formula for encryption



```
[11/11/26]seed@VM:~/.../rsa$ nano task2.c
[11/11/26]seed@VM:~/.../rsa$ gcc task2.c -lcrypto -o task2 && ./task2
Encrypted Ciphertext (C): 6FB078DA550B2650832661E14F4F8D2CFAEF475A0DF3A75CACDC5DE5CFC5FADC
[11/11/26]seed@VM:~/.../rsa$ █
```

encrypted message

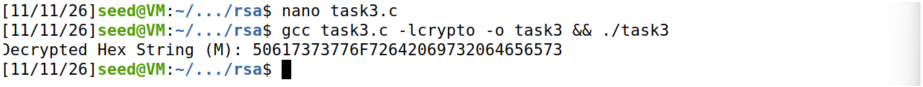
### Task 3: Decrypting a Message

The goal is to decrypt the cipher text and to convert the resulting Hex back into ASCII String.

```
C = 8C0F971DF2F3672B28811407E2DABBE1DA0FEBBBDFC7DCB67396567EA1E2493F
```

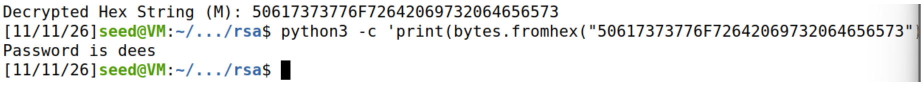
The ciphertext

```
// Compute M = C^d mod n
    BN_mod_exp(M, C, d, n, ctx);
```



```
[11/11/26]seed@VM:~/.../rsa$ nano task3.c
[11/11/26]seed@VM:~/.../rsa$ gcc task3.c -lcrypto -o task3 && ./task3
)ecrypted Hex String (M): 50617373776F72642069732064656573
[11/11/26]seed@VM:~/.../rsa$ █
```

Decrypted Hex String



```
Decrypted Hex String (M): 50617373776F72642069732064656573
[11/11/26]seed@VM:~/.../rsa$ python3 -c 'print(bytes.fromhex("50617373776F72642069732064656573")
Password is dees
[11/11/26]seed@VM:~/.../rsa$ 
```

Hex converted to ASCII String

### Task 4: Signing a Message

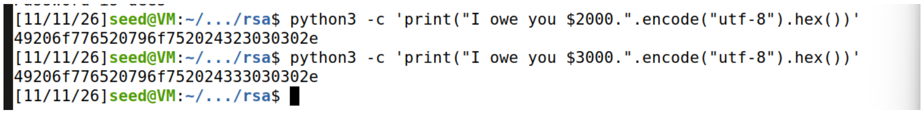
Digital signatures are used to prove authenticity. They provide: 1. Authenticity: verifying who signed the message 2. Integrity: detects whether the message has been changed 3. Non-repudiation: provides evidence that the holder of the private key signed the message

`S = H(M)^d mod n`

H(M) is the hashed message

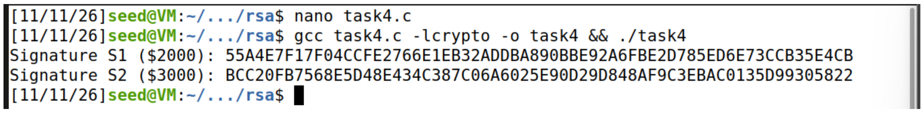
`n = DCBFFE3E51F62E09CE7032E2677A78946A849DC4CDDE3A4D0CB81629242FB1A5`
`e = 010001 (this hex value equals to decimal 65537)`
`M = A top secret!`
`M1 = I owe you $2000.`
`M2 = I owe you $3000.`
`d = 74D806F9F3A62BAE331FFE3F0A68AFE35B3D2E4794148AACBC26AA381CD7D30D`

The message and the public/private keys



```
[11/11/26]seed@VM:~/.../rsa$ python3 -c 'print("I owe you $2000.".encode("utf-8").hex())'
49206f776520796f75202432303030302e
[11/11/26]seed@VM:~/.../rsa$ python3 -c 'print("I owe you $3000.".encode("utf-8").hex())'
49206f776520796f75202433303030302e
[11/11/26]seed@VM:~/.../rsa$ 
```

Hex version of the messages before signing



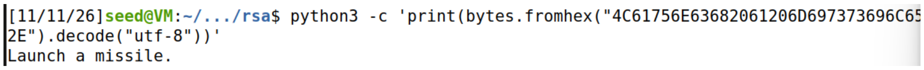
```
[11/11/26]seed@VM:~/.../rsa$ nano task4.c
[11/11/26]seed@VM:~/.../rsa$ gcc task4.c -lcrypto -o task4 && ./task4
Signature S1 ($2000): 55A4E7F17F04CCFE2766E1EB32ADDBA890BBE92A6FBE2D785ED6E73CCB35E4CB
Signature S2 ($3000): BCC20FB7568E5D48E434C387C06A6025E90D29D848AF9C3EBAC0135D99305822
[11/11/26]seed@VM:~/.../rsa$ 
```

Message 1 and 2 are signed very differently despite changing 1 value. It is unidentifiable.

### Task 5: Verifying a Signature

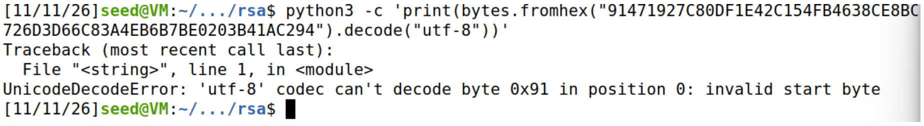
Bob received the message `Launch a missile.` with a signature S. The goal is to verify if it is from Alice

`M = Launch a missile.`
`S = 643D6F34902D9C7EC90CB0B2BCA36C47FA37165C0005CAB026C0542CBDB6802F`
`e = 010001 (this hex value equals to decimal 65537)`
`n = AE1CD4DC432798D933779FBD46C6E1247F0CF1233595113AA51B450F18116115`



```
[11/11/26]seed@VM:~/.../rsa$ python3 -c 'print(bytes.fromhex("4C61756E63682061206D697373696C65
2E").decode("utf-8"))'
Launch a missile.
```

The signature decodes to the intital message



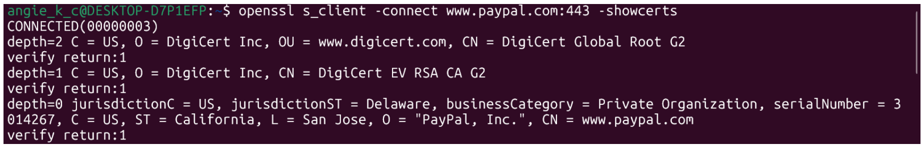
```
[11/11/26]seed@VM:~/.../rsa$ python3 -c 'print(bytes.fromhex("91471927C80DF1E42C154FB4638CE8BC
726D3D66C83A4EB6B7BE0203B41AC294").decode("utf-8"))'
Traceback (most recent call last):
  File "<string>", line 1, in <module>
UnicodeDecodeError: 'utf-8' codec can't decode byte 0x91 in position 0: invalid start byte
[11/11/26]seed@VM:~/.../rsa$
```

Corrupting the last bit results in a UnicodeDecodeError because of invalid bits.

## Task 6: Manually Verifying an X.509 Certificate

**Step 1: Download a certificate from a real web server** We downloaded a certificate, Paypal's.

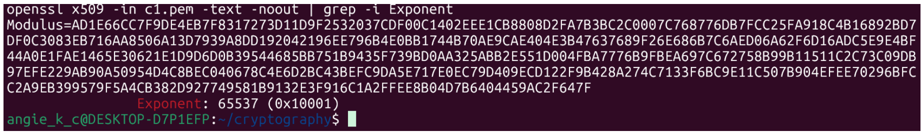
`openssl s_client -connect www.paypal.com:443 -showcerts`



```
angie_k_c@DESKTOP-D7P1EFP:~$ openssl s_client -connect www.paypal.com:443 -showcerts
CONNECTED(00000003)
depth=2 C = US, O = DigiCert Inc, OU = www.digicert.com, CN = DigiCert Global Root G2
verify return:1
depth=1 C = US, O = DigiCert Inc, CN = DigiCert EV RSA CA G2
verify return:1
depth=0 jurisdictionC = US, jurisdictionST = Delaware, businessCategory = Private Organization, serialNumber = 3
014267, C = US, ST = California, L = San Jose, O = "PayPal, Inc.", CN = www.paypal.com
verify return:1
```

Figure 1: alt text

**Step 2: Extract the public key (e, n) from the issuer's certificate** Extracting (e, n) from the issuer's certificate



```
openssl x509 -in c1.pem -text -noout | grep -i Exponent
Modulus=AD1E66CC7F9DE4EB7F8317273D11D9F2532037CDF00C1402EEE1CB8808D2FA7B3BC2C0007C768776DB7FCC25FA918C4B16892BD7
DF0C3083EB716AA8506A13D7939A8DD192042196EE796B4E0BB1744B70AE9CAE404E3B47637689F26E686B7C6AED06A62F6D16ADC5E9E4BF
44A0E1FAE1465E30621E1D9D6D0B39544685BB751B9435F739BD0AA325ABB2E551D004FBA7776B9FBEA697C672758B99B11511C2C73C09DB
97EFE229AB90A50954D4C8BEC040678C4E6D2BC43BEFC9DA5E717E0EC79D409ECD122F9B428A274C7133F6BC9E11C507B904EFEE70296BFC
C2A9EB399579F5A4CB382D927749581B9132E3F916C1A2FFEE8B04D7B6404459AC2F647F
                Exponent: 65537 (0x10001)
angie_k_c@DESKTOP-D7P1EFP:~/cryptography$
```

Figure 2: alt text

```
AD1E66CC7F9DE4EB7F8317273D11D9F2532037CDF00C1402EEE1CB8808D2FA7B3BC2C0007C768776DB7FCC25FA91
```

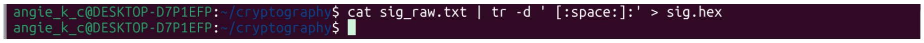
Value of n

```
 65537
```

Value of e

**Step 3: Extracting the signature from the server's certificate** When the SHA256 hashed signature is extracted, it has colons and spaces which need to be cleaned.

`openssl x509 -in c0.pem -text -noout`



```
angie_k_c@DESKTOP-D7P1EFP:~/cryptography$ cat sig_raw.txt | tr -d ' [:space:]:' > sig.hex
angie_k_c@DESKTOP-D7P1EFP:~/cryptography$
```

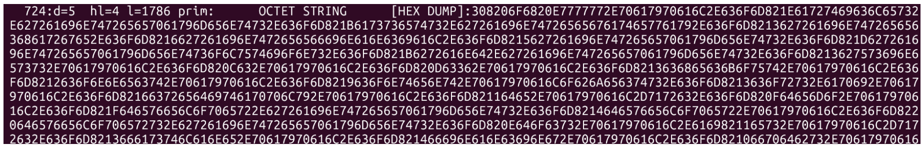
Figure 3: alt text

```
2503824558e14cbd175c2030b31601ba8a6d6412f79d17027f55ab2c12297239da83390921e5fe22636c9ff68c5
```

Cleaned signature

**Step 4: Extract the body of the server's certificate** We are extracting the body so that we can compare it with the results gotten after verification of the signature

```
openssl asn1parse -i -in c0.pem
openssl asn1parse -i -in c0.pem -strparse 4 -out c0_body.bin -noout
sha256sum c0_body.bin
```

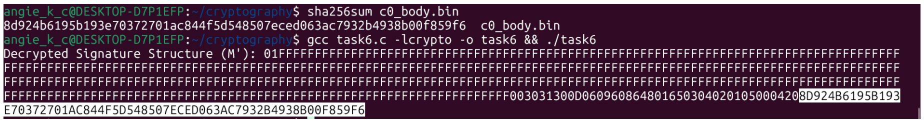


```
  724:d=5  hl=4 l=1786 prim:      OCTET STRING      [HEX DUMP]:308206F6820E7777772E70617970616C2E636F6D821E61727469636C65732
E627261696E747265657061796D656E74732E636F6D821B6173736574732E627261696E74726565676174657761792E636F6D8213627261696E747265656
368617267652E636F6D8216627261696E7472656566696E616E6369616C2E636F6D8215627261696E747265657061796D656E74732E636F6D821D6272616
96E747265657061796D656E74736F6C7574696F6E732E636F6D821B6272616E642E627261696E747265657061796D656E74732E636F6D8213627573696E6
573732E70617970616C2E636F6D820C632E70617970616C2E636F6D820D63362E70617970616C2E636F6D8213636865636B6F75742E70617970616C2E636
F6D8212636F6E6E6563742E70617970616C2E636F6D8219636F6E74656E742E70617970616C6F626A656374732E636F6D8213636F72732E6170692E70617
970616C2E636F6D82166372656469746170706C792E70617970616C2E636F6D821164652E70617970616C2D7172632E636F6D820F64656D6F2E706179706
16C2E636F6D821F646576656C6F7065722E627261696E747265657061796D656E74732E636F6D8214646576656C6F7065722E70617970616C2E636F6D822
0646576656C6F706572732E627261696E747265657061796D656E74732E636F6D820E646F63732E70617970616C2E6169821165732E70617970616C2D717
2632E636F6D8213666173746C616E652E70617970616C2E636F6D821466696E616E63696E672E70617970616C2E636F6D821066706462732E70617970616
```

Figure 4: alt text

**Step 5: Verify the signature** We have the (e, n) and the signature. We will compare the output here with that from the body

The message is usually hash hence the sha256sum on the content from the body.



```
angie_k_c@DESKTOP-D7P1EFP:~/cryptography$ sha256sum c0_body.bin
8d924b6195b193e70372701ac844f5d548507eced063ac7932b4938b00f859f6  c0_body.bin
angie_k_c@DESKTOP-D7P1EFP:~/cryptography$ gcc task6.c -lcrypto -o task6 && ./task6
Decrypted Signature Structure (M'): 01FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF003031300D0609608648016503040201050004208D924B6195B193
E70372701AC844F5D548507ECED063AC7932B4938B00F859F6
```

Figure 5: alt text